Initiation au Développement : Algorithmique Fondamentale, Variables et Structures de Contrôle

Thème principal : Variables en RAM, Affectation Destructive, Structures Conditionnelles et Boucles Iteratives

Synthèse exécutive : Ce cours pose les fondations absolues de l'algorithmique et du développement logiciel. Il résout le problème de la traduction d'un processus décisionnel ou calculatoire du monde réel en une séquence formelle, explicite et déterministe d'instructions exécutables par un processeur. La séance traite de la modélisation de la mémoire vive (RAM) à travers le quadruplet d'une variable (Nom, Type, Valeur, Adresse), de la mécanique d'affectation destructive, du contrôle du flux d'exécution via les prédicats booléens et les structures conditionnelles (Si / Alors / Sinon), ainsi que de l'automatisation des traitements répétitifs au moyen de boucles bornées (Pour) et non bornées (TantQue).

1. Mémorisation et Manipulation des Données : Les Variables et la Mémoire RAM

En informatique, le concept de variable se distingue radicalement de sa définition mathématique statique. Il ne s'agit pas d'une inconnue abstraite dans une équation, mais d'une case mémoire physique concrète située dans la RAM de l'ordinateur. Une variable est obligatoirement caractérisée par un quadruplet : son Nom (qui permet au développeur de la manipuler), son Type (qui indique au système le nombre d'octets à réserver et le format de codage binaire), sa Valeur (le contenu actuel à un instant t) et son Adresse mémoire physique (emplacement hexadécimal en RAM). Lors de l'exécution d'une instruction d'affectation (ex: x ← x + 1), l'Unité Centrale de Traitement (CPU) applique un protocole en deux étapes chronologiques : premièrement, elle lit en mémoire RAM les valeurs des variables figurant à droite du symbole d'affectation et évalue l'expression globale ; deuxièmement, elle vient inscrire ce résultat final dans la case mémoire associée à la variable située à gauche. Ce processus est strictement destructif : le nouveau résultat écrase définitivement l'ancienne valeur. Le piège majeur en partiel consiste à utiliser une variable sans l'avoir initialisée. Si un algorithme cherche à lire une variable non initialisée dans un calcul, le système va récupérer les bits résiduels restés en mémoire vive (la 'valeur poubelle'), engendrant des calculs faux ou des comportements totalement imprévisibles du programme.

- Variable : Emplacement mémoire étiqueté réservé dans la mémoire vive (RAM) destiné à conserver une valeur modifiable au cours de l'exécution.
- Quadruplet d'une variable : Ensemble indissociable constitué par son Nom (identificateur), son Type (format et taille en octets), sa Valeur actuelle et son Adresse physique en RAM.
- Typage formel : Spécification explicite du domaine de valeurs (Entier, Réel, Chaîne de caractères, Booléen) déterminant la quantité de mémoire allouée et les opérations permises.
- Affectation (← ou =) : Opération destructive d'écriture qui évalue l'expression située à droite et remplace sans retour la valeur stockée à gauche.
- Valeur poubelle : Donnée binaire résiduelle non nettoyée présente en RAM, lue lorsqu'une variable est exploitée sans avoir été préalablement initialisée.

Cas concret enseignant : *La métaphore de l'armoire d'archivage à tiroirs : Imaginez la mémoire RAM comme une grande armoire d'archivage contenant des tiroirs. Déclarer une variable revient à coller une étiquette sur un tiroir (ex: 'solde_compte') et à dimensionner sa profondeur selon le type ('Entier'). L'instruction d'affectation 'solde_compte ← 150' consiste à ouvrir ce tiroir précis, à jeter au recyclage le morceau de papier qui s'y trouvait auparavant (l'ancienne valeur), et à y déposer une nouvelle feuille portant l'inscription 150.*

2. Prise de Décision : Les Structures Conditionnelles (Si / Alors / Sinon)

Par défaut, l'ordinateur exécute un programme de manière séquentielle, de la première à la dernière instruction, ligne après ligne. Pour doter l'algorithme d'un comportement adaptatif et intelligent, on introduit des structures conditionnelles. Lorsqu'un bloc 'Si (Condition) Alors [Bloc A] Sinon [Bloc B]' est rencontré, le processeur évalue

l'expression booléenne. Si le prédicat donne la valeur VRAI, l'ordinateur exécute exclusivement le [Bloc A], puis saute par-dessus le [Bloc B] pour reprendre le flux normal. Si la condition donne la valeur FAUX, le [Bloc A] est ignoré et seul le [Bloc B] (s'il existe) est exécuté. Pour formaliser des critères décisionnels complexes, la maîtrise des tables de vérité est requise : l'opérateur ET (AND) exige que l'ensemble des sous-conditions soient simultanément vraies pour valider le test global ; l'opérateur OU (OR) requiert qu'au moins une des sous-conditions soit vraie ; l'opérateur NON (NOT) inverse la valeur de vérité de l'expression.

- Structure d'Alternative (Si / Alors / Sinon) : Branchement logique détournant le flux d'exécution séquentiel selon la valeur de vérité d'une condition.
- Expression Booléenne (Prédicat) : Proposition logique évaluée par le processeur qui ne peut prendre que deux états strictement binaires : VRAI (True) ou FAUX (False).
- Opérateurs Logiques (ET, OU, NON) : Connecteurs booléens permettant de composer des conditions complexes régies par leurs tables de vérité respectives.

Cas concret enseignant : *Le cas réel du Distributeur Automatique de Billets (DAB) : Avant d'expulser les billets de banque, l'automate évalue une condition composée : SI (solde_client >= montant_demandé) ET (billets_disponibles_dans_automate >= montant_demandé). Si cette expression globale est VRAIE, le paiement est validé et les billets sont délivrés ; SINON, l'opération est refusée, un message d'erreur est affiché sur l'écran et la carte bancaire est restituée à l'usager.*

3. Répétition et Automatisation : Les Structures Itératives (Boucles Pour et TantQue)

La véritable puissance des systèmes informatiques réside dans leur capacité à exécuter des séries d'instructions répétitives à très haute vitesse et sans altération. Deux structures itératives principales sont utilisées en algorithmique : 1. La boucle POUR (déterministe) : Elle est privilégiée lorsque le nombre d'itérations est connu avant d'entrer dans la boucle. Elle intègre nativement l'initialisation d'une variable compteur, le test de borne supérieure et l'incrémentation systématique à chaque itération (ex: traiter les N éléments d'un tableau). 2. La boucle TANT QUE (indéterminée) : Elle s'impose lorsque le nombre de répétitions dépend d'un événement dynamique au cours du programme (ex: valider la saisie d'un mot de passe utilisateur). La condition est évaluée AVANT chaque passage. Si elle s'avère vraie, le corps de la boucle s'exécute ; si elle est fausse au départ, le corps n'est jamais exécuté. La règle d'or d'une boucle TANT QUE est de modifier impérativement, à l'intérieur du corps de la boucle, au moins une variable engagée dans la condition de sortie. Si la condition reste indéfiniment VRAIE, l'algorithme tombe dans une boucle infinie qui bloque l'exécution et sature le processeur.

- Boucle Pour (FOR) : Structure itérative bornée et déterministe, utilisée lorsque le nombre précis de répétitions est connu à l'avance.
- Boucle TantQue (WHILE) : Structure itérative non bornée et dynamique, répétée tant qu'une condition logique demeure VRAIE.
- Variable de Contrôle & Incrémentation : Modification explicite d'un compteur au sein du corps de la boucle pour garantir le franchissement de la condition de sortie.
- Boucle Infinie : Dysfonctionnement critique lié à l'absence d'actualisation de la condition de sortie dans une boucle TantQue, gelant le programme et monopolisant les ressources CPU.

Cas concret enseignant : *La métaphore de l'entraîneur d'athlétisme : Si l'entraîneur ordonne au coureur 'Fais 10 tours de piste', il s'agit d'une boucle POUR (le nombre 10 est connu dès le départ et la fin est garantie après 10 tours). S'il ordonne 'Cours tant qu'il ne pleut pas', il s'agit d'une boucle TANT QUE (la sortie dépend d'une observation météo réévaluée à chaque tour de piste).*

■ Questions & Interventions des Étudiants en Classe

Question étudiant : Pourquoi l'instruction 'x = x + 1' (ou 'x ← x + 1') est-elle valide en programmation alors qu'elle est mathématiquement impossible ?
Réponse du professeur : En mathématiques, le signe égal '=' représente une égalité statique entre deux expressions. En informatique, il s'agit d'une opération d'affectation dynamique et séquentielle. Le processeur évalue d'abord le côté droit (x + 1) en lisant la valeur actuelle de x stockée en RAM, puis vient inscrire le résultat obtenu dans la case mémoire x, écrasant destructivement l'ancienne valeur.

Question étudiant : Que se passe-t-il concrètement si j'oublie de modifier la variable de contrôle à l'intérieur du corps d'une boucle TantQue ?
Réponse du professeur : Si la condition d'entrée de la boucle était vraie au départ, elle le restera éternellement puisque rien ne change la valeur des variables testées dans le corps de la boucle. Le programme entre alors dans une 'boucle infinie', consommant 100% des ressources du processeur et bloquant l'exécution du reste de l'application.

■ Exercice & Démonstration d'Application

Énoncé : Écrire un algorithme en pseudo-code qui demande à l'utilisateur de saisir un nombre entier positif N, puis calcule et affiche la somme accumulée de tous les entiers compris entre 1 et N inclus (Somme = 1 + 2 + ... + N).

Résolution pas à pas :
Étape 1 : Déclaration explicite des variables avec typage : - N : Entier (limite saisie par l'utilisateur) - i : Entier (compteur / variable de contrôle de boucle) - somme : Entier (accumulateur) Étape 2 : Interaction et saisie utilisateur : Afficher "Veuillez saisir un nombre entier positif N : " Lire N Étape 3 : Initialisation stricte de l'accumulateur : somme ← 0 // Indispensable pour purger la mémoire RAM de toute valeur poubelle Étape 4 : Structure itérative bornée de calcul (Boucle Pour) : Pour i allant de 1 à N par pas de 1 Faire somme ← somme + i FinPour Étape 5 : Restitution du résultat final : Afficher "La somme des entiers de 1 à ", N, " est égale à : ", somme

Résultat : Pour une saisie utilisateur N = 5, le suivi pas à pas en mémoire RAM est le suivant : - Initialisation : somme = 0 - i = 1 → somme ← 0 + 1 = 1 - i = 2 → somme ← 1 + 2 = 3 - i = 3 → somme ← 3 + 3 = 6 - i = 4 → somme ← 6 + 4 = 10 - i = 5 → somme ← 10 + 5 = 15 Le programme affiche : 'La somme des entiers de 1 à 5 est égale à : 15'. L'algorithme est déterministe et valide.

■ Auto-Évaluation Flash d'Examen

Q : Quelle est la différence fondamentale de critère de choix entre une boucle POUR et une boucle TANT QUE ?

R : La boucle POUR est une structure itérative bornée utilisée lorsque le nombre exact de répétitions est connu avant le début de l'exécution (déterministe). La boucle TANT QUE est une structure non bornée utilisée lorsque le nombre de répétitions dépend d'une condition dynamique évaluée avant chaque itération.

Q : Que valent les variables A et B après l'exécution exacte du bloc d'instructions suivant : A ← 5 ; B ← 3 ; A ← A + B ; B ← A - B ?

R : Après 'A ← 5' et 'B ← 3', on a A=5 et B=3. Après 'A ← A + B', A prend la valeur 5 + 3 = 8. Enfin, après 'B ← A - B', B prend la valeur 8 - 3 = 5. À la fin de l'exécution, A vaut 8 et B vaut 5 (cela réalise un échange de valeurs).

Q : Pourquoi l'absence d'initialisation d'une variable d'accumulation (ex: somme ← 0) fait-elle perdre l'intégralité des points en examen ?

R : Sans initialisation explicite, la variable contient une 'valeur poubelle' (données binaire résiduelles non nettoyées en RAM). Lors de la première addition (somme ← somme + i), le calcul utilise cette valeur indéterminée, ce qui fausse le résultat final et rend l'algorithme totalement imprévisible.